

ADVANCED AI AUTOMATION

Engineer's Roadmap: LangChain · Agentic AI · n8n · CRM Integrations

10 weeks · for developers who already know React, Next.js, and LLM APIs

Who this roadmap is for

You already know React, Next.js, TypeScript, and have called LLM APIs before. You understand fetch, async/await, and REST. This roadmap skips all of that and goes straight to what you don't have yet: orchestrating AI into real systems — agents that reason, pipelines that run autonomously, and CRM integrations that actually save time.

What you'll be able to build by the end

- Multi-agent LangGraph systems where AI nodes reason, delegate, and self-correct
- RAG pipelines with vector search, hybrid retrieval, and citation-aware answers
- Self-hosted n8n on your own VPS with HTTPS, Postgres persistence, and production uptime
- Automated CRM workflows on HubSpot and Zoho (both free) — lead scoring, enrichment, follow-ups
- A CI/CD pipeline with GitHub Actions that deploys your agents and n8n configs automatically
- A full capstone product: agentic sales intelligence platform with observability and evals

The five phases

PHASE 1 LangChain foundations — chains, memory, tools · Weeks 1–2

PHASE 2 Agentic AI — RAG, LangGraph, multi-agent systems · Weeks 3–4

PHASE 3 Self-hosted n8n + CI/CD pipeline · Weeks 5–6

PHASE 4 Free CRM integrations — HubSpot & Zoho · Weeks 7–8

PHASE 5 Capstone — full agentic product in production · Weeks 9–10

Tools and cost

- LangChain + LangGraph
 - Free and open source. Use OpenRouter (free credits) or GroqCloud (free tier) as LLM providers.
- n8n
 - Self-hosted = free forever. You pay only for the VPS (~\$4–6/month on Hetzner or DigitalOcean).
- Supabase
 - Free tier includes PostgreSQL + pgvector. Generous limits for development and small production apps.
- HubSpot CRM
 - Free forever plan — contacts, deals, pipelines, email, forms. No credit card. Full REST API access.
- Zoho CRM
 - Free for up to 3 users — leads, contacts, deals, workflows. REST API accessible on free plan.

- LangSmith
 - Free developer tier — traces, evals, prompt versioning. Essential for debugging agents in production.
- GitHub Actions CI/CD
 - Free for public repos and 2,000 minutes/month on private. More than enough.
- Tavily Search API
 - Free tier: 1,000 searches/month. The best search tool for LangChain agents.

Prerequisites

- React + Next.js — comfortable with API routes, server components, fetch
- TypeScript — not expert level, but you can read and write typed code
- LLM APIs — you've called OpenAI or similar before, understand messages array, temperature, etc.
- Git — commits, branches, push. Week 5's CI/CD builds on top of this.
- Basic terminal comfort — cd, ls, running npm scripts, SSH into a server

The guiding principle: agents are just functions that call LLMs. Everything else — LangChain, LangGraph, n8n, CRM integrations — is about making those functions composable, observable, and automatic. Never lose sight of that.

You already know how to call an LLM API. LangChain is about what to do after that — how to chain calls, structure outputs, give the model tools, and manage memory across turns. This phase covers the modern LCEL (LangChain Expression Language) approach, not the legacy chain classes you'll see in old tutorials.

WEEK 1 · the LCEL pipeline pattern

Chains, Prompts & Structured Outputs

The core insight of LangChain is that everything is a Runnable — a thing you can pipe into another thing. PromptTemplate | LLM | OutputParser is a chain. You compose these with the | operator (LCEL). This week you internalize that pattern completely.

WHAT TO LEARN

- **LCEL — LangChain Expression Language**
 - The pipe operator (|) composes Runnables. PromptTemplate | model | StrOutputParser() is your simplest chain.
 - RunnableSequence, RunnableParallel, RunnableLambda — three building blocks you'll use everywhere.
 - stream() vs invoke() vs batch() — streaming is what users expect; batch is what production pipelines use.
- **PromptTemplate and ChatPromptTemplate**
 - Difference between a system message, human message, and AI message. Structure your prompts as message arrays, not raw strings.
 - Partial prompts — pre-fill some variables, leave others dynamic. Useful for system prompts with static context.
- **Output parsers — getting structured data out of LLMs**
 - StrOutputParser for raw text, JsonOutputParser for JSON, and most importantly: StructuredOutputParser with Zod schemas.
 - withStructuredOutput() — the cleanest way to force structured output. Attaches a tool call schema to the model.
 - PydanticOutputParser equivalent in JS: use Zod to define the schema, pass to withStructuredOutput.
- **Model providers and routing**
 - ChatOpenAI works with any OpenAI-compatible endpoint. Point it at OpenRouter or GroqCloud with baseUrl override.
 - Free models via OpenRouter: mistralai/mistral-7b-instruct, meta-llama/llama-3-8b-instruct, google/gemma-7b-it.
 - Free models via GroqCloud: llama-3.1-8b-instant, mixtral-8x7b-32768. Much faster inference than OpenRouter.
- **Sequential chains — when output of one becomes input of the next**
 - Pipe the output of one chain into the next. RunnableSequence handles this. Use RunnablePassthrough to pass-through original input alongside transformed output.

SETUP

- npm install langchain @langchain/core @langchain/openai
 - @langchain/core is the base. langchain is the main package. @langchain/openai is the model provider.
- Create a .env.local in your Next.js project: OPENROUTER_API_KEY, GROQ_API_KEY, LANGCHAIN_API_KEY (for LangSmith)
- Sign up for LangSmith now (smith.langchain.com — free dev tier) and set LANGCHAIN_TRACING_V2=true
 - Every chain call will now appear in LangSmith with full prompt/response/latency. Invaluable for debugging.

PRACTICE EXERCISES

- Build a 3-step sequential chain: extract keywords → expand each into a sentence → format as a bullet list
- Build a parallel chain: summarise a text AND extract sentiment simultaneously, then merge both into one response
- Force JSON output with withStructuredOutput + a Zod schema with 5+ nested fields

▶ Week 1 Project — Job Application Intelligence Pipeline

[Major] A Next.js app that processes a raw job description and a candidate's resume, runs them through a 4-step LangChain pipeline, and produces a scored match report with a tailored cover letter draft.

Requirements:

- Chain 1 — Job parser: extract structured data from JD using withStructuredOutput + Zod: { role, company, requiredSkills[], niceToHave[], salaryRange, remotePolicy }
- Chain 2 — Resume parser: same approach for the candidate's resume text: { skills[], experience[], education[] }
- Chain 3 — Matcher: RunnableParallel that runs skill gap analysis and culture fit assessment simultaneously
- Chain 4 — Generator: takes match scores + JD + resume → writes a personalised cover letter in the candidate's voice
- Next.js API route at /api/analyze that accepts POST with { jobDescription, resumeText }
- React UI: two textareas for input, a 'Analyze' button, and a structured results card showing match score breakdown + cover letter
- Stream Chain 4's output to the UI using Vercel AI SDK's useChat or a plain ReadableStream
- LangSmith tracing enabled — open the LangSmith dashboard and walk through the full trace of one run
- All model calls go through GroqCloud (llama-3.1-8b-instant) — free and fast enough for this pipeline

Why this project: Forces you to use every LCEL concept together: sequential chains, parallel chains, structured output, streaming, and a real UI. The pipeline pattern repeats in every project after this.

Stack: [LangChain](#) • [LCEL](#) • [@langchain/openai](#) • [GroqCloud \(free\)](#) • [Zod](#) • [Next.js](#) • [LangSmith](#)

W E E K 2 • the ReAct loop and tool calling

Memory, Tools & Your First Agent

Memory makes the LLM conversational. Tools make it capable. An agent is what you get when you give a model memory + tools and let it decide which tool to call based on a user's request. This week you build agents from scratch, then use LangChain's AgentExecutor to manage the loop.

WHAT TO LEARN

- Conversation memory types
 - ConversationBufferMemory — stores every message. Fine for short sessions, hits context limits fast.
 - ConversationSummaryMemory — uses a second LLM call to compress old messages into a summary. Better for long sessions.
 - ConversationBufferWindowMemory — keeps only the last N turns. Simple and predictable.
 - Where to store memory: in-memory (dev only), Supabase (prod) — use SupabaseVectorStore for semantic memory.
- Tool calling — giving the LLM abilities
 - DynamicTool: define a name, description, and a function. The description is what the LLM reads to decide when to use it.
 - Tool descriptions matter more than the function code. Bad description = agent never uses the tool. Good description = agent uses it correctly.
 - Built-in tools: TavilySearchResults (web search), Calculator, WikipediaQueryRun.
- The ReAct pattern — how agents think

- Thought → Action → Observation → Thought → ... → Final Answer. The agent writes its reasoning before every action.
- AgentExecutor manages this loop. Set verbose: true in dev to see each thought/action/observation in the terminal.
- maxIterations — always set this. An uncapped agent loop is a credit card bill waiting to happen.
- Creating tools that talk to your own systems
 - Write a DynamicTool that queries Supabase. The agent can now look things up in your database.
 - Write a DynamicTool that calls an external API (REST Countries, Open-Meteo). The agent can now fetch live data.

TAVILY SETUP

- Sign up at tavily.com — free tier gives 1,000 searches/month. Get your API key.
- npm install @langchain/community — Tavily tool lives here: `import { TavilySearchResults } from '@langchain/community/tools/tavily_search'`

▶ Week 2 Project — Conversational Research Agent with Live Tools

[Major] A Next.js chat interface where an AI agent answers questions using web search, a Supabase knowledge base, and a calculator. The agent shows its reasoning process in the UI.

Requirements:

- Agent tools: TavilySearchResults (web), a custom Supabase lookup tool (searches a 'knowledge_base' table you create), a Calculator tool
- ConversationSummaryMemory backed by GroqCloud — compresses history when it exceeds 2,000 tokens
- AgentExecutor with maxIterations: 6, verbose: true. Log each step to the server console.
- Next.js API route: POST /api/agent — accepts { message, sessionId }. Loads memory for that sessionId from Supabase.
- Streaming: stream the agent's final answer token-by-token to the UI using TransformStream
- React UI: chat window with message history + a collapsible 'Agent reasoning' panel that shows Thought/Action/Observation steps
- Seed the Supabase knowledge_base table with 10 rows of domain-specific facts so the lookup tool returns something meaningful
- LangSmith: view the full agent trace — every tool call, every thought. Screenshot and save for your portfolio.
- Bonus: add a 'Clear memory' button that deletes the session's memory from Supabase

Why this project: Building an agent with real tools connected to real data is the pattern you'll reuse in every automation project. The streaming reasoning UI is impressive to show clients and employers.

Stack: LangChain AgentExecutor • Tavily (free) • Supabase • ConversationSummaryMemory • GroqCloud • Next.js streaming

Phase 1 built single agents. Phase 2 builds systems — RAG pipelines that search your own knowledge, and multi-agent graphs where specialist agents collaborate. LangGraph is the framework; it treats your agent as a state machine rather than a loop, giving you precise control over routing, branching, and human approval gates.

WEEK 3 · vector search, chunking, hybrid retrieval

RAG — Retrieval Augmented Generation

RAG is how you give an LLM knowledge of your own documents without fine-tuning. Chunk the docs, embed them into a vector database, retrieve the most relevant chunks at query time, inject them into the prompt. The quality of your chunking and retrieval strategy determines 80% of RAG quality.

WHAT TO LEARN

- Document loading and chunking
 - pdf-parse for PDFs, cheerio for web pages, fs.readFile for plain text. All become LangChain Document objects.
 - RecursiveCharacterTextSplitter: splits by paragraph → sentence → word → character. Best general-purpose splitter.
 - Chunk size vs overlap tradeoff: 500–1000 chars per chunk, 100–200 char overlap. Too small = no context, too large = noise.
 - Semantic chunking (experimental): splits by semantic meaning, not character count. Higher quality, higher cost.
- Embeddings and vector storage with Supabase pgvector
 - Enable pgvector extension in Supabase: CREATE EXTENSION IF NOT EXISTS vector; in the SQL editor.
 - Create a table: documents (id, content, metadata jsonb, embedding vector(1536)).
 - OpenAI's text-embedding-3-small: 1536 dimensions, cheap (~\$0.02/million tokens), excellent quality.
 - SupabaseVectorStore.fromDocuments() — chunks → embed → insert. One call handles the full pipeline.
- Retrieval strategies
 - Similarity search (cosine): find the N chunks whose embedding is closest to the query embedding. Default approach.
 - MMR (Maximum Marginal Relevance): balances similarity with diversity — avoids returning 5 chunks that say the same thing.
 - Hybrid search: combine vector similarity with full-text keyword search using Supabase's ts_vector. Much better recall for specific terms.
 - Re-ranking: take top 20 results from retrieval, re-rank with a cross-encoder model, keep top 5. LangChain has CohereRerank for this.
- Prompt engineering for RAG
 - Always include: 'Answer based only on the following context. If the answer is not in the context, say so.'
 - Citations: ask the LLM to include source document IDs in its answer. Parse them and link to the original document.
 - The stuffing approach vs map-reduce vs refine — for large context windows, stuffing is usually best.

EVALUATING RAG QUALITY

- Build a test set: 20 question/answer pairs where you know the correct answer
- Measure: retrieval recall (did the right chunk get retrieved?), answer faithfulness (did the LLM stick to the context?)
- LangSmith has built-in RAG evaluation tools — use them

▶ Week 3 Project — Document Q&A System with Citations

[Major] A Next.js app where users upload PDFs, which get chunked, embedded, and stored in Supabase pgvector. Users ask questions in a chat interface and get cited answers with links back to source pages.

Requirements:

- File upload UI: accept PDF files, send to Next.js API route /api/ingest
- Ingest pipeline: pdf-parse → RecursiveCharacterTextSplitter (800 chars, 150 overlap) → text-embedding-3-small → SupabaseVectorStore
- Store metadata: { filename, pageNumber, uploadedAt } on each chunk. This powers citations.
- Query API route /api/query: embed the question → hybrid search (vector + keyword) → retrieve top 5 chunks → stuff into prompt
- System prompt forces citation: 'Cite sources as [filename, page X]. Only answer from context.'
- withStructuredOutput for the answer: { answer: string, citations: Array<{ filename, page, quote }> }
- React chat UI: renders citations as collapsible cards below each AI answer showing the exact quote and page number
- MMR retrieval — compare answer quality vs plain similarity search on 10 test questions
- LangSmith eval: create a dataset of 15 Q&A pairs, run evals, record faithfulness and relevance scores
- Bonus: add a document management page showing all ingested PDFs with chunk counts and delete option

Why this project: RAG is the most commonly requested AI feature in real products. Citations make RAG trustworthy enough for business use. This project is immediately demonstrable and deployable.

Stack: Supabase pgvector • text-embedding-3-small • pdf-parse • SupabaseVectorStore • LangSmith evals • Next.js

WEEK 4 • supervisor patterns, human-in-the-loop, persistence

LangGraph — Multi-Agent State Machines

LangGraph treats your agent workflow as a directed graph. Each node is a function. Edges are routes — some conditional (IF the agent chose tool X, go to node Y). State is an object that every node reads from and writes to. This architecture gives you far more control than AgentExecutor, and it scales to complex multi-agent systems.

WHAT TO LEARN

- LangGraph core concepts
 - StateGraph: define your state shape (TypeScript interface), add nodes, add edges, compile.
 - Nodes: plain async functions that take state as input and return a partial state update.
 - Edges: route between nodes. Conditional edges use a function that inspects state and returns the next node name.
 - START and END: the entry and exit nodes. Every graph starts at START and ends when it reaches END.
- Checkpointers — persisting graph state
 - A checkpointer saves graph state between turns so you can resume a conversation or pause for approval.
 - MemorySaver for dev (in-memory). For production: write a custom checkpointer that saves to Supabase.
 - thread_id: a string that identifies a conversation. Pass it in config to load the right checkpoint.
- Human-in-the-loop — pausing for approval
 - interrupt_before: ['node_name'] — the graph pauses before running that node. Your code can inspect state and decide to approve or cancel.

- `graph.invoke(null, { configurable: { thread_id } })` — resume a paused graph by calling `invoke` with the same `thread_id`.
- This is how you build approval flows: agent drafts an email → pauses → human approves → agent sends it.
- **Supervisor / worker multi-agent pattern**
 - Supervisor node: reads the task, decides which worker agent to call next, routes via conditional edge.
 - Worker agents: each has its own tools and specialisation. Researcher, Writer, Critic, Publisher are common roles.
 - Workers report back to supervisor after completing their task. Supervisor decides: done, or needs another worker?
- **Streaming LangGraph execution to the UI**
 - `graph.stream()` yields events as each node completes. Each event has: `{ [nodeName]: stateUpdate }`.
 - Stream these via Server-Sent Events (SSE) from a Next.js API route to the React frontend.

▶ **Week 4 Project — Multi-Agent Content Factory**

[Major] A LangGraph system with 5 specialist agents orchestrated by a supervisor. Given a topic, it researches, drafts, critiques, rewrites, and publishes content — with a human approval gate before publishing.

Requirements:

- StateGraph with state: `{ topic, research, draft, critique, finalDraft, approved, published, messages[] }`
- Supervisor node: decides which worker to route to next based on current state
- Researcher agent: uses TavilySearchResults (3 searches), synthesises findings into structured research notes
- Writer agent: takes research notes + topic → writes a 400-word blog post with H2 subheadings
- Critic agent: evaluates the draft on 5 criteria (accuracy, clarity, SEO, tone, length) → returns scores + specific rewrite instructions
- Rewriter agent: takes draft + critique → produces improved final draft
- Human approval node: graph pauses here. Next.js UI shows the final draft with Approve/Reject buttons.
- Publisher node (only runs after approval): saves to Supabase 'posts' table with title, content, `research_sources[]`, `published_at`
- SSE streaming: stream each node completion to the React UI — users watch the agents work in real time
- Supabase checkpoint: persist graph state between server restarts so in-progress workflows survive deploys
- LangSmith: full multi-agent trace visible — every supervisor decision, every worker output

Why this project: *Multi-agent systems with human-in-the-loop approval are what businesses actually deploy. This is the architecture behind most real AI products — not a single chatbot, but a network of specialists with a human staying in control.*

Stack: [@langchain/langgraph](#) • [StateGraph](#) • [Supabase checkpoint](#) • [TavilySearchResults](#) • [SSE streaming](#) • [LangSmith](#)

n8n is where your AI agents become automation systems that run without you. Self-hosting on a VPS gives you full control — no usage limits, no data leaving your infrastructure, persistent workflows. Week 5 is infrastructure. Week 6 is building production automation pipelines and wiring in your LangChain agents. CI/CD in week 6 means every push to GitHub auto-deploys your agents and n8n configs.

WEEK 5 · Docker, Caddy, Postgres, production setup

Self-Hosted n8n on a VPS

This is the most infrastructure-heavy week in the roadmap. Do it right once and you won't touch it again. The goal: n8n running at your own domain with HTTPS, backed by Postgres, containerised with Docker, auto-restarting on crash, monitored for uptime.

VPS SETUP

- Get a VPS — Hetzner CX11 (€3.79/mo, 2GB RAM, 20GB SSD) or DigitalOcean Basic Droplet (\$6/mo)
 - Hetzner is cheaper and just as reliable. Choose Ubuntu 22.04 LTS. Pick a datacenter near your users.
 - Add an SSH key during setup — never use password auth on a production server.
- Initial server hardening
 - Create a non-root user: `adduser deploy && usermod -aG sudo deploy`
 - Disable root SSH: `edit /etc/ssh/sshd_config → PermitRootLogin no → systemctl restart ssh`
 - UFW firewall: `ufw allow 22 && ufw allow 80 && ufw allow 443 && ufw enable`
 - Keep it updated: `apt update && apt upgrade -y` on first login
- Install Docker + Docker Compose
 - Follow the official Docker install docs for Ubuntu. Install docker-compose-plugin (not the old docker-compose binary).
 - Add your deploy user to docker group: `usermod -aG docker deploy`. Log out and back in.
- Point your domain to the VPS
 - Add an A record in your DNS: `n8n.yourdomain.com → your VPS IP`. Wait for propagation (~5 min on Cloudflare).

N8N DOCKER COMPOSE CONFIGURATION

- Create `/home/deploy/n8n/docker-compose.yml` with three services: n8n, postgres, caddy
 - n8n service: image `n8nio/n8n`, environment variables for DB connection, `WEBHOOK_URL`, encryption key, timezone.
 - Critical env vars: `N8N_HOST`, `WEBHOOK_URL=https://n8n.yourdomain.com`, `N8N_ENCRYPTION_KEY` (generate with `openssl rand -hex 32`), `DB_TYPE=postgresdb`, `DB_POSTGRESDB_DATABASE`, `DB_POSTGRESDB_USER`, `DB_POSTGRESDB_PASSWORD`.
 - postgres service: image `postgres:15`, named volume for data persistence. Never use SQLite for production n8n.
 - caddy service: image `caddy:2-alpine`, Caddyfile that reverse proxies `n8n.yourdomain.com` to `n8n:5678` with auto-HTTPS.
- Caddyfile (the entire thing is 3 lines)
 - `n8n.yourdomain.com { reverse_proxy n8n:5678 }` — Caddy auto-gets and renews the TLS cert from Let's Encrypt.
- `docker compose up -d` — all three containers start. Check logs: `docker compose logs -f n8n`
- Set restart: unless-stopped on all services — they auto-restart if the server reboots

MONITORING AND MAINTENANCE

- UptimeRobot (free) — monitors your n8n URL every 5 minutes, emails you if it goes down
- Docker log rotation — add logging: `{ driver: json-file, options: { max-size: '10m', max-file: '3' } }` to each service

- Backups — add a daily cron job that `pg_dump` the Postgres database and uploads to Supabase Storage or S3-compatible storage
- Updating n8n — `docker compose pull` & `docker compose up -d` — takes 30 seconds

N8N BASICS ON YOUR SELF-HOSTED INSTANCE

- Visit `n8n.yourdomain.com`, create your admin account (this is the only account — it's not a SaaS)
- Settings → Community Nodes: install `n8n-nodes-base` (already included). Explore available nodes.
- Build 3 test workflows before moving on: webhook echo, scheduled Supabase ping, HTTP Request to GroqCloud

📌 Week 5 Project — Production n8n Infrastructure

[Major] A fully running self-hosted n8n instance at your own domain with HTTPS, Postgres persistence, monitoring, and automated database backups. This is infrastructure as a deliverable.

Requirements:

- n8n running at `n8n.yourdomain.com` with a valid TLS certificate (green padlock in browser)
- Docker Compose file with n8n + Postgres + Caddy. All services have `restart: unless-stopped`.
- Postgres database persisting workflow data — verify by creating a workflow, restarting all containers, confirming workflow survived
- UptimeRobot monitor sending email alerts to your email — test by temporarily stopping n8n and confirming alert arrives
- Automated daily backup: bash script (`pg_dump` → `gzip` → upload to Supabase Storage) running via cron at 2am
- Test webhook: create a Webhook trigger workflow in n8n, copy the URL, POST to it from Postman and from curl, confirm execution
- Test scheduled workflow: every 5 minutes, call Open-Meteo for Rawalpindi weather, save temperature to Supabase table 'weather_log'
- SSH hardening: confirm you can only login via SSH key, root login disabled, UFW active with only ports 22/80/443 open
- Write a one-page runbook: how to SSH in, restart containers, check logs, restore from backup, update n8n — your future self will thank you

Why this project: Infrastructure work is unglamorous but permanent. A well-configured VPS runs unattended for months. Every automation you build in weeks 6–10 depends on this foundation being solid.

Stack: Hetzner/DigitalOcean VPS • Docker Compose • Caddy (auto-HTTPS) • Postgres persistence • UptimeRobot (free) • `pg_dump` backups

WEEK 6 • calling LangChain from n8n, GitHub Actions deployment

Agent Endpoints + CI/CD Pipeline

Your LangChain agents live in a Next.js app. n8n needs to call them. You'll build clean API endpoints that wrap your agents, then create a CI/CD pipeline that auto-deploys both your Next.js app (Vercel) and your n8n workflows (via n8n's CLI) whenever you push to main. Push code → tests run → deploy happens → n8n workflows update. No manual steps.

CALLING YOUR AGENTS FROM N8N

- Build Next.js API routes that wrap your LangChain agents from Phase 1 and 2
 - `POST /api/agents/pipeline` — the job application pipeline from Week 1
 - `POST /api/agents/research` — the research agent from Week 2
 - `POST /api/agents/rag` — the RAG Q&A system from Week 3

- Each endpoint: authenticate with a shared secret header (X-Agent-Key), invoke the agent, stream or return the result
- In n8n: HTTP Request node → your Vercel endpoint → parse JSON response → route to next node
 - Store the X-Agent-Key in n8n Credentials (Header Auth type). Never hardcode secrets in workflows.
 - Add error handling: if HTTP status ≥ 400 , route to an error branch that logs to Supabase and sends you a Slack message.
- n8n Code node — when built-in nodes aren't enough
 - Runs Node.js. Access items (the data from the previous node) and return transformed items.
 - Use it for JSON transformations, date math, conditional logic that's too complex for the IF node.

CI/CD PIPELINE WITH GITHUB ACTIONS

- GitHub Actions: YAML files in `.github/workflows/` that run on push, PR, or schedule
 - Every push to main triggers the workflow. Steps run sequentially in a clean Ubuntu container.
- Workflow: on push to main → run tests → deploy to Vercel → export n8n workflows → SSH to VPS and import
 - Step 1 (Test): `npm ci && npm test`. If tests fail, deployment stops. Your agents are tested before deployment.
 - Step 2 (Deploy Next.js): use Vercel's GitHub Action (`vercel/action`). Needs `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID` in GitHub Secrets.
 - Step 3 (Deploy n8n workflows): use `appleboy/ssh-action` to SSH into your VPS, then run n8n CLI to import updated workflow JSON files from your repo.
- n8n workflow-as-code: export your workflows as JSON, commit them to the repo
 - `n8n export:workflow --all --output=./n8n-workflows/` exports all workflows as JSON files.
 - Commit these JSON files. In CI: SSH to VPS → `n8n import:workflow --input=./n8n-workflows/` — workflows update automatically.
- Environment secrets in GitHub: Settings → Secrets → Actions
 - `VERCEL_TOKEN`, `VPS_SSH_KEY`, `VPS_HOST`, `VPS_USER`, `OPENROUTER_API_KEY`, `LANGCHAIN_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`
 - These are injected into the workflow as `${{ secrets.SECRET_NAME }}`. Never log secret values.
- Branch protection: require CI to pass before merging to main
 - GitHub → Settings → Branches → Add rule → Require status checks. Enforces: no broken code reaches production.

🎯 Week 6 Project — AI Lead Enrichment Pipeline with Full CI/CD

[Major] An automated lead enrichment system: n8n receives a new lead via webhook, calls your LangChain research agent, saves enriched data to Supabase, and posts a summary to Slack. The entire stack deploys automatically via GitHub Actions on every push to main.

Requirements:

- Next.js API route `/api/agents/enrich`: takes `{ name, company, email }` → LangChain agent uses Tavily to research the company → returns structured enrichment: `{ companySize, industry, techStack[], linkedinSummary, potentialPainPoints[], fitScore }`
- n8n workflow: Webhook trigger (someone fills a lead form) → HTTP Request to `/api/agents/enrich` → Parse JSON → Supabase node (insert to 'leads' table) → Slack node (post enriched summary to #leads channel)
- Error handling workflow: if any node fails → log error to Supabase 'errors' table → Slack alert to #alerts channel
- GitHub Actions workflow file (`.github/workflows/deploy.yml`): on push to main → npm test → Vercel deploy → SSH to VPS → n8n workflow import
- Write at least 3 unit tests for your enrichment agent: test structured output schema, test error handling when Tavily returns no results, test fit scoring logic
- Commit your n8n workflow JSON to the repo. Modify the workflow locally, export, commit,

push — confirm CI automatically updates the live n8n workflow

- Branch protection enabled: the CI pipeline must pass before any PR can merge to main
- Demonstrate the full loop: make a code change → push → watch GitHub Actions → see updated agent live on Vercel → see updated workflow in n8n
- Slack integration: create a free Slack workspace, add an Incoming Webhook, add the webhook URL to n8n Credentials

Why this project: *CI/CD is what separates hobby projects from professional software. Every company expects this. The n8n workflow-as-code pattern means your automations are version controlled, reviewable, and deployable like any other code.*

Stack: [GitHub Actions](#) • [Vercel deploy action](#) • [appleboy/ssh-action](#) • [n8n CLI \(import/export\)](#) • [n8n Webhook](#) • [Slack node](#) • [Supabase](#)

Both HubSpot (free forever plan) and Zoho CRM (free up to 3 users) have full REST APIs accessible on their free tiers. This phase builds AI-powered workflows on top of both — lead scoring, enrichment, automated follow-up sequences, and two-way sync between CRMs and your Supabase data layer.

WEEK 7 · contacts, deals, pipelines, email sequences

HubSpot CRM API + AI Lead Workflows

HubSpot's free CRM is the most developer-friendly free CRM available. It has a clean REST API, detailed documentation, and a free Private App mechanism for authentication. You can manage contacts, deals, companies, and email threads programmatically without paying anything.

HUBSPOT FREE PLAN COVERS

- Contacts (unlimited on free), Companies, Deals, Tasks, Notes — full CRUD via API
- Pipelines and deal stages — automate moving deals through stages based on AI scores
- Email: log emails to contact timelines via the Engagements API
- Forms and Webhooks: embed a HubSpot form, get notified on submission — your n8n trigger
- Custom Properties: add any custom field to contacts or deals — use for AI scores, enrichment data, `last_enriched_at`

HUBSPOT API SETUP

- Create a free HubSpot account at hubspot.com — no credit card
- Settings → Integrations → Private Apps → Create Private App
 - Scopes you need: `crm.objects.contacts (read/write)`, `crm.objects.deals (read/write)`, `crm.schemas.contacts (read)`, `timeline (write)`
 - Copy the generated access token. Add to n8n as a Header Auth credential: Authorization: Bearer YOUR_TOKEN
- Base URL: `https://api.hubapi.com` — all endpoints are under `/crm/v3/objects/`
- Test in Postman first — GET `/crm/v3/objects/contacts` to see your existing contacts

KEY HUBSPOT API PATTERNS

- Create a contact: POST `/crm/v3/objects/contacts` with `{ properties: { email, firstname, lastname, company } }`
- Search contacts: POST `/crm/v3/objects/contacts/search` with `filters` array — find by email, company, custom property
- Update a contact: PATCH `/crm/v3/objects/contacts/{contactId}` — add your AI score as a custom property
- Create a deal and associate it with a contact: POST `/crm/v3/objects/deals + associations API`
- HubSpot webhooks: Settings → Integrations → Webhooks — subscribe to `contact.creation`, `deal.propertyChange` events

N8N HUBSPOT NODE VS HTTP REQUEST NODE

- n8n has a built-in HubSpot node — good for simple CRUD (create contact, update deal). Use it for standard operations.
- For search, batch operations, or newer API endpoints: use HTTP Request node — more flexible, same credentials

[Major] When a new contact is created in HubSpot (via webhook), n8n triggers: your LangChain agent enriches the contact, scores their fit 1-10, and automatically updates HubSpot with the score, enrichment data, and moves the deal to the correct pipeline stage.

Requirements:

- HubSpot webhook → n8n: subscribe to contact.creation. New contact triggers the n8n workflow.
- n8n Node 1: HTTP Request — GET contact details from HubSpot using the contactId from the webhook payload
- n8n Node 2: HTTP Request → /api/agents/enrich (your Week 6 agent endpoint) — enrich with company research
- n8n Node 3: HTTP Request → /api/agents/score — new LangChain chain: takes enrichment data → returns { score: 8, reasoning: '...', recommendedAction: 'schedule_call' }
- n8n Node 4: HubSpot node — update contact with custom properties: hs_lead_score, enrichment_summary, ai_recommended_action, enriched_at
- n8n Node 5: IF node — score >= 7 → create a Deal and associate with contact, set pipeline stage to 'Qualified'. Score < 7 → add tag 'nurture'.
- n8n Node 6: HubSpot Engagements API — log a note to the contact timeline: 'AI enrichment completed. Score: 8/10. Reason: [reasoning]'
- Test with 5 different contact types: enterprise, SMB, startup, freelancer, student. Review scores and reasoning in HubSpot.
- Build a simple Next.js dashboard that reads the enriched leads from Supabase (also sync there) and displays them sorted by AI score
- CI/CD: this workflow is committed as JSON to your repo and auto-deploys via your Week 6 GitHub Actions pipeline

Why this project: Lead scoring is one of the highest-ROI automation workflows in B2B. Every sales team wants this. Having it built, deployed, and connected to a real CRM is a portfolio piece that gets hired.

Stack: [HubSpot API \(free\)](#) · [HubSpot webhooks](#) · [n8n HubSpot node](#) · [LangChain scoring chain](#) · [Supabase sync](#) · [GitHub Actions deploy](#)

W E E K 8 · two-way sync, abstraction, follow-up sequences

Zoho CRM + Multi-CRM Patterns

Zoho CRM's free plan supports up to 3 users and exposes a REST API. Critically, Zoho's API architecture is different from HubSpot's — different auth (OAuth2 vs Bearer), different data models (Leads vs Contacts, Potentials vs Deals). This week teaches CRM-agnostic patterns: abstract your workflows so the same AI logic works across both CRMs.

ZOHO CRM FREE PLAN COVERS

- Leads, Contacts, Accounts, Deals (Potentials), Tasks, Calls — all via REST API
- Workflows and automation triggers — limited on free, but webhooks work fine via Deluge or direct API
- Custom fields on any module — add your AI score and enrichment fields here
- Zoho Flow (limited free tier) or direct webhook notification to n8n via Zoho Functions

ZOHO API SETUP — OAUTH2 (MORE COMPLEX THAN HUBSPOT)

- Sign up at zoho.com/crm — free for up to 3 users. No credit card.
- Register an API client at api-console.zoho.com → Self Client → get Client ID and Client Secret
- OAuth2 flow: exchange credentials for access_token + refresh_token. Access tokens expire in 1 hour — use refresh_token to get new ones.

- In n8n: create an OAuth2 Generic credential with Zoho's token endpoint. n8n handles token refresh automatically.
- Base URL: <https://www.zohoapis.com/crm/v2/> — note: different region = different URL (EU, IN, AU, JP)

CRM-AGNOSTIC ABSTRACTION PATTERN

- Build a CRM adapter layer in your Next.js app: a set of functions with a consistent interface regardless of which CRM is being used
 - `createContact(data)` → internally calls either HubSpot or Zoho based on a config flag
 - `updateContact(id, data)`, `createDeal(data)`, `logNote(contactId, text)` — same interface, different implementations
 - In n8n: a single workflow handles both CRMs. The first node reads from Supabase to know which CRM this contact lives in.
- Unified contact schema in Supabase
 - 'contacts' table: `id`, `email`, `name`, `company`, `crm_source` ('hubspot' | 'zoho'), `crm_id`, `ai_score`, `enrichment` jsonb, `last_synced_at`
 - This is your source of truth. CRMs are views into this data, not the source.
- Two-way sync patterns
 - CRM → Supabase: webhook on CRM update → n8n → upsert Supabase record
 - Supabase → CRM: n8n schedule (every 15 min) checks for Supabase records updated since last sync → pushes to CRM
 - Conflict resolution: last-write-wins with a `updated_at` timestamp comparison. Supabase wins if both updated in same window.
 - Deduplication: before creating, always search by email first. If found, update. If not, create.

📺 Week 8 Project — AI Follow-Up Sequence Engine (Dual CRM)

[Major] A scheduled n8n workflow that runs every morning, finds contacts in both HubSpot and Zoho who haven't been followed up with in 3+ days, generates a personalised AI follow-up email for each, sends it via the CRM's email API, and logs the interaction.

Requirements:

- Supabase query: `contacts WHERE last_contacted_at < now() - interval '3 days' AND ai_score >= 5 AND status != 'closed'`
- For each contact: HTTP Request → `/api/agents/followup` — new LangChain chain that takes { `contact_data`, `enrichment`, `last_interaction_note` } → returns personalised email { `subject`, `body` } in the contact's industry tone
- CRM routing: IF `crm_source == 'hubspot'` → HubSpot Engagements API to send email + log. IF `crm_source == 'zoho'` → Zoho Email API + log.
- After sending: update Supabase `last_contacted_at`, increment `follow_up_count`, log the email body to `contact_interactions` table
- Deduplication guard: check `last_contacted_at` WITHIN the workflow before sending — prevents race conditions if the workflow runs twice
- Rate limiting: add a Wait node (2 second delay between contacts) to avoid hitting CRM API rate limits
- Daily summary email: at end of workflow, aggregate results (X emails sent, Y HubSpot, Z Zoho, N failed) → send summary to yourself via Resend (free tier email API)
- Dashboard: Next.js page showing today's sent follow-ups, which contacts, which CRM, AI-generated email preview
- All deployed via CI/CD — push workflow JSON, GitHub Actions updates n8n automatically

Why this project: Personalised follow-up at scale is what sales teams spend hours on every day. This system does it automatically and better — the AI uses actual enrichment data, not a generic template. Two-CRM support makes it immediately applicable to real client work.

Stack: Zoho CRM API (free) • HubSpot API (free) • Supabase two-way sync • LangChain follow-up chain • Resend email API (free) • n8n scheduling

Everything you've built connects into one coherent product. The capstone is a real, deployed, production-grade AI system: an autonomous sales intelligence platform that researches companies, scores leads, writes outreach, syncs to both CRMs, and runs as a 24/7 automated pipeline. Week 9 adds observability, evals, and production hardening. Week 10 is the capstone build.

WEEK 9 · LangSmith in depth, error budgets, prompt versioning

Observability, Evals & Production Hardening

Most AI projects fail in production not because the model is bad, but because nobody is watching what it does. This week you add proper observability to everything you've built — traces, evals, alerts, and prompt versioning. This is what separates a demo from a product.

LANGSMITH IN DEPTH

- **Projects and tracing:** every agent run is a trace. Group by project in LangSmith (one project per feature).
 - `trace.metadata`: add custom metadata to every run — { `userId`, `contactId`, `workflow`, `version` }. Lets you filter traces in LangSmith.
 - Tags: tag runs with model name, prompt version, and environment (dev/prod). Filter by tag in the dashboard.
- **Building an eval dataset**
 - Create a LangSmith Dataset: 25 input/output pairs for each agent. Input = what the agent receives, output = what a correct response looks like.
 - Run evaluations via the LangSmith SDK: `client.runOnDataset({ datasetName, llmOrChain, evaluators })`
 - Built-in evaluators: criteria evaluation (is the output helpful? accurate? concise?), exact match, regex match.
 - Custom evaluator: write a function that grades your agent's structured output against the expected schema.
- **Prompt versioning in LangSmith Hub**
 - Push prompts to LangSmith Hub: `hub.push('my-org/lead-scorer-v1', prompt)`. Pull by version: `hub.pull('my-org/lead-scorer-v1:abc123')`.
 - Never change a production prompt in code without creating a new Hub version. Tag versions as 'production', 'staging', 'experiment'.
 - A/B test prompts: run the same eval dataset against two prompt versions, compare scores. Promote the winner to production.
- **Production hardening: fallbacks, retries, circuit breakers**
 - Model fallback: `.withFallbacks([primaryModel, fallbackModel])`. If GroqCloud is down, fall back to OpenRouter.
 - Retry with backoff: wrap model calls in a retry utility. 3 retries, exponential backoff (1s, 2s, 4s).
 - Timeout: every agent call should have a maximum execution time. Abort after 30 seconds rather than hanging forever.
 - Cost tracking: log token usage from every LLM call to Supabase. Alert via n8n if daily spend exceeds threshold.
- **n8n error monitoring**
 - Settings → Error Workflows: one global error handler workflow that catches all failures across all workflows.
 - Log to Supabase: { `workflow_name`, `node_name`, `error_message`, `input_data`, `timestamp` }. Build a simple error dashboard.
 - Alert thresholds: if more than 3 errors in 1 hour from the same workflow, send a Slack alert with priority level.



Week 9 Project — Full Observability Stack

[Medium] Add production-grade observability to all agents from Phases 1–4. Build an internal monitoring dashboard showing agent health, error rates, eval scores, and prompt versions.

Requirements:

- LangSmith project setup: separate projects for pipeline, research, RAG, enrichment, scoring, follow-up agents
- Add custom metadata to every agent call: `run_id`, `workflow_name`, `contact_id`, `model`, `prompt_version`
- Eval datasets: 15 Q&A pairs for the RAG agent, 15 lead-score inputs with expected outputs for the scoring agent
- Run evals via LangSmith SDK in a GitHub Actions nightly job: if eval scores drop below threshold, open a GitHub Issue automatically
- Prompt Hub: move all production prompts to LangSmith Hub. Update agents to pull prompts by version at runtime.
- Model fallback: all agents fall back to OpenRouter if GroqCloud returns a 5xx error
- n8n error dashboard: Next.js page that reads from the Supabase errors table — shows error rate by workflow, last 24h errors, most common failure nodes
- Cost tracking: `log { model, prompt_tokens, completion_tokens, estimated_cost_usd }` for every LLM call. Weekly cost report via n8n email.
- Load test one agent endpoint with k6 (free) — find the breaking point, document it, add a rate limiter

Why this project: *You cannot improve what you cannot measure. This week makes your entire system debuggable, auditable, and safe to run with real client data. This is what companies mean when they say 'production-ready AI'.*

Stack: LangSmith (evals + hub) • GitHub Actions nightly eval job • k6 load testing (free) • n8n error workflow • Supabase cost/error tables

WEEK 10 • everything connected, deployed, and running

Capstone — Autonomous Sales Intelligence Platform

The capstone is not a tutorial project. It's a real product that you design, build, and deploy. It connects everything from all 9 weeks into one cohesive system. By the end, you have a live URL, a running n8n instance automating real tasks, agents connected to real CRMs, full observability, and a CI/CD pipeline that keeps it all deployed and updated.

SYSTEM ARCHITECTURE

- Next.js app (deployed on Vercel): dashboard + agent API endpoints + RAG upload interface
- LangGraph multi-agent system: Researcher → Scorer → Copywriter → Reviewer → Approver (human gate) → Publisher
- n8n (self-hosted VPS): orchestrates scheduling, CRM sync, email sequences, error handling
- Supabase: companies table, contacts table, outreach_history, research_cache, agent_logs, errors
- HubSpot + Zoho: bi-directionally synced via n8n. AI scores visible in both CRMs.
- LangSmith: full observability on every agent. Eval suite running nightly.
- GitHub Actions CI/CD: push to main → test → deploy Next.js → update n8n workflows



Week 10 Capstone — Autonomous Sales Intelligence Platform

[Capstone] A complete, deployed, production-grade AI system. Input a list of target companies. The system autonomously researches, scores, writes personalised outreach, syncs to HubSpot and Zoho, runs follow-up sequences, and shows everything in a live dashboard.

Requirements:

- DASHBOARD (Next.js / Vercel): upload CSV of target companies → triggers the LangGraph pipeline → live progress tracking via SSE → results table with AI scores, research summaries, outreach drafts
- LANGGRAPH PIPELINE: Researcher (Tavily, 3 searches per company) → Scorer (1-10 fit + confidence) → Copywriter (personalised cold email per contact) → Reviewer (critique + rewrite) → Human Approval Gate (dashboard button) → Publisher (saves to DB, creates CRM records)
- RAG INTEGRATION: each company's research is embedded into pgvector. Copywriter queries RAG to find relevant context before writing. Follow-up agent also queries RAG before writing follow-ups.
- N8N AUTOMATION (self-hosted): Schedule: daily 9am → query Supabase for contacts needing follow-up → AI follow-up generation → send via CRM email API → log interaction
- N8N CRM SYNC: HubSpot webhook → n8n → update Supabase. Zoho webhook → n8n → update Supabase. Schedule: every 30 min → check Supabase for CRM drift → push corrections.
- N8N ALERTS: error workflow → Slack alert. Cost alert if daily LLM spend > \$2. Eval score drop alert if nightly eval < 70%.
- CI/CD: GitHub Actions deploys Next.js to Vercel + imports n8n workflow JSONs to VPS on every push to main. PR checks: type-check + unit tests + LangSmith eval must pass.
- OBSERVABILITY: LangSmith traces all agent runs. Nightly eval job tests 25 scoring + 15 RAG cases. Prompt versions tracked in LangSmith Hub. All errors logged to Supabase + Slack.
- DOCUMENTATION: Write a 1-page architecture decision record (ADR) explaining why you chose each tool. Document the API endpoints. Write setup instructions someone else could follow.
- DEMO: record a 5-minute screen recording showing: upload companies → agents run → outreach appears → CRM records created → n8n follow-up fires → LangSmith trace walkthrough

Why this project: *This is a real product, not a tutorial. It solves a real problem (sales research and outreach at scale), uses every skill from the roadmap, runs in production, and can be shown to potential clients or employers. It is your proof of work.*

Stack: LangGraph • LangSmith • n8n (self-hosted) • HubSpot + Zoho (free) • Supabase pgvector • GitHub Actions • Vercel • Next.js • Tailwind

Stack acquired.

10 weeks ago you had React skills. Now you have a production AI system running on your own infrastructure, connected to real CRMs, with observability, CI/CD, and a capstone project you can show anyone. These are the tools being used in production at every AI-forward company right now.

What this stack unlocks

- Freelance/agency work: businesses pay \$3K-15K for the kind of CRM automation + AI enrichment workflows you now know how to build
- Product work: any AI product team needs engineers who understand agents, RAG, and production hardening — you now do
- Your own products: you have the full stack to build and deploy your own AI SaaS — agents, database, automation, CRM

Where to go from here

- Explore LangGraph's Persistence API — multi-tenant agent state management at scale
- Voice agents: Retell AI + your LangChain agents = a phone-call AI. You already understand the webhook pattern it uses.
- MCP (Model Context Protocol) — the emerging standard for connecting agents to external tools. LangChain is adding native support.
- OpenAI Assistants API — alternative to LangGraph for simpler agent state. Good for product comparison knowledge.
- Deploy n8n workflows to customers: a white-label n8n instance per client is a real agency model.

Full tool reference

- LangChain docs: js.langchain.com — always check LCEL and integrations sections first
- LangGraph docs: langchain-ai.github.io/langgraphjs — StateGraph API reference is essential
- LangSmith: smith.langchain.com — traces, evals, hub all documented here
- n8n docs: docs.n8n.io — self-hosting, node reference, workflow patterns
- HubSpot API: developers.hubspot.com — CRM API reference, webhook docs
- Zoho CRM API: www.zoho.com/crm/developer — OAuth guide, modules reference
- Supabase pgvector: supabase.com/docs/guides/ai — embedding, vector search, hybrid search
- GitHub Actions: docs.github.com/actions — workflow syntax, secrets, deployment actions
- GroqCloud: console.groq.com — free API keys, model list, rate limits
- OpenRouter: openrouter.ai/docs — model list, free credits, OpenAI-compatible API
- Tavily: docs.tavily.com — search API, LangChain integration